

Função

listen_for_incoming_data()

```
def listen_for_incoming_data():
    global memoria_gps

    packet = hardware_receive(timeout=0.1, max_bytes=15)

    if packet is not None:
        # Para testar COM encriptação, usar a linha de baixo:
        # decrypted_packet = decrypt_aes_ctr(packet)

        # Para testar SEM encriptação, usar a linha de baixo:
        decrypted_packet = packet

        # Mede o tamanho exato do pacote para saber para onde o encaminhar
        tamanho = len(decrypted_packet)

        if tamanho == 15:
            dados = unpack_telemetria_completa(decrypted_packet)

            if dados:
                vest_id = dados['vest_id']

                lat_graus_calc = dados['lat_raw'] // 100000
                lat_min_calc = abs(dados['lat_raw']) % 10000000

                lon_graus_calc = dados['lon_raw'] // 100000
                lon_min_calc = abs(dados['lon_raw']) % 10000000
```

0000

```
        memoria_gps[vest_id] = {
            'lat_graus': lat_graus_calc,
            'lat_min': lat_min_calc,
            'lon_graus': lon_graus_calc,
            'lon_min': lon_min_calc
        }

        print(f"[Colete {vest_id}] Trama Completa |
BPM: {dados['bpm']} | Temp: {dados['temperature']:.2f}°C")
        return dados

    elif tamanho == 7:

        vest_id = decrypted_packet[0] & 0x1F

        lat_min_novo, lon_min_novo = struct.unpack('>HH', decrypted_packet[1:5])

        if vest_id in memoria_gps:
            estado = memoria_gps[vest_id]

            limite_superior = 9000000 # Perto de 10 mil
hões
            limite_inferior = 1000000 # Perto de zero

            # Lógica de Rollover da Latitude:
            # Se estava num valor alto e caiu para um m
uito baixo, andou 1 Grau para a frente
            if estado['lat_min'] > limite_superior and
lat_min_novo < limite_inferior:
                estado['lat_graus'] += 1
            # Se estava num valor baixo e saltou para u
m muito alto, andou 1 Grau para trás
            elif estado['lat_min'] < limite_inferior an
d lat_min_novo > limite_superior:
                estado['lat_graus'] -= 1
```

```

        estado['lat_min'] = lat_min_novo # Atualiza
os decimais novos

        # Repete a Lógica de Rollover para a Longit
ude
        if estado['lon_min'] > limite_superior and
lon_min_novo < limite_inferior:
            estado['lon_graus'] += 1
        elif estado['lon_min'] < limite_inferior an
d lon_min_novo > limite_superior:
            estado['lon_graus'] -= 1
            estado['lon_min'] = lon_min_novo

        print(f"[GPS DELTA] Colete {vest_id} -> {es
tado['lat_graus']}.{estado['lat_min']}")

        return {
            "tipo": "DELTA_GPS",
            "vest_id": vest_id,
            "lat": f"{estado['lat_graus']}.{estado
['lat_min']}",
            "lon": f"{estado['lon_graus']}.{estado
['lon_min']}"
        }

    elif tamanho == 4:

        vest_id = decrypted_packet[0] & 0x1F
        flags = decrypted_packet[1]

        sos = bool((flags >> 6) & 0x01)
        md = bool((flags >> 5) & 0x01)

        if sos or md:
            tipo = "SOS" if sos else "MAN DOWN"
            print(f"!!! ALERTA CRÍTICO: Colete {vest_i
d} em {tipo} !!!")

```

```

        return {
            "tipo": "ALERTA",
            "vest_id": vest_id,
            "sos": sos,
            "man_down": md
        }

    elif tamanho == 2:
        vest_id = decrypted_packet[0] & 0x1F
        opcode = (decrypted_packet[1] >> 4) & 0x0F

        # Se o opcode for 9 (0b1001), é o Colete a confirmar que ouviu o Dashboard
        print(f"[ACK] Colete {vest_id} confirmou a recepção da mensagem.")

        return {
            "tipo": "ACK",
            "vest_id": vest_id,
            "status": "RECEBIDO"
        }

    return None # Se o pacote chegou estragado ou com um tamanho inválido, ignora

```

```

def listen_for_incoming_data():
    global memoria_gps

    packet = hardware_receive(timeout=0.1, max_bytes=15)

```

- **global memoria_gps:** Esta função vai precisar de atualizar a localização de todos os coletes no mapa, o que esta linha faz é pedir autorização para poder alterar.
- **hardware_receive(...):** Fica à escuta na porta UART (ou LoRa) durante o tempo definido. Se chegar alguma coisa (máximo de 15 bytes), guarda

no packet.

```
if packet is not None:
    decrypted_packet = packet
    tamanho = len(decrypted_packet)
```

- Aqui primeiro verificamos se o pacote é vazio.
- **tamanho = len(...):** O que esta linha faz é medir o pacote para no código abaixo decidir para que **if** a mensagem vai ser enviada.

```
if tamanho == 15:
    dados = unpack_telemetria_completa(decrypted_packet)
```

- Se o tamanho for de 15 bytes, então é uma Trama Completa. O que fazemos é chamar a função **unpack_telemetria_completa(...)** para desempacotar.

```
lat_graus_calc = dados['lat_raw'] // 10000000
lat_min_calc = abs(dados['lat_raw']) % 10000000
```

- Como no "*NetworkArchitectureSpecification*" diz "Graus decimais multiplicados por 10^7 " então aqui vamos ter que fazer o oposto. Primeiro fazemos a divisão inteira por 10^7 (**// 10000000**), aqui extraímos apenas os graus (ex: 38) depois fazemos o resto da divisão (**% 10000000**) de onde tiramos as casas decimais (ex: 7500000).
- Isto é guardado na **memoria_gps[vest_id]**.

```
elif tamanho == 7:
    vest_id = decrypted_packet[0] & 0x1F
    lat_min_novo, lon_min_novo = struct.unpack('>HH', decrypted_packet[1:5])
```

- Como o colete só envia a Trama Completa de 10 em 10 minutos. Nos intervalos, é enviada apenas a Trama de Posição(delta) que tem o tamanho de 7 bytes. Esta trama não contém os graus só tem os decimais do GPS, porque os graus não estão constantemente a mudar logo não ha necessidade de estar sempre a enviá-los.

```
if estado['lat_min'] > limite_superior and lat_min_novo <
limite_inferior:
    estado['lat_graus'] += 1
```

- Um dos problemas de não enviar os graus era no caso em que por exemplo o bombeiro estava na coordenada 38.9999999 e dava um passo para a frente para a coordenada 39.0000001. Para resolver isso foi implementado o seguinte:
- Se a memória antiga (estado['lat_min']) estava perto do valor máximo(ex: 9.900.000) e a nova leitura (lat_min_novo) está perto do mínimo (ex: 100.000), então isso significa que o bombeiro mudou o grau. Então no código, somamos +1 Grau à memória.
- Para o inverso, a lógica é a “mesma” (-= 1) (caso em que ele andar para trás).

```
elif tamanho == 4:
    vest_id = decrypted_packet[0] & 0x1F
    flags = decrypted_packet[1]
    sos = bool((flags >> 6) & 0x01)
    md = bool((flags >> 5) & 0x01)
```

- Esta trama tem prioridade máxima. Se o tamanho for 4, o código vai logo ler as *Flags*(alertas).
- O operador >> 6 é usado para ler o bit do (**SOS**) e o operador >> 5 é usado para ler o bit de **Man Down** (se o bombeiro caiu no chão).

```
elif tamanho == 2:  
    vest_id = decrypted_packet[0] & 0x1F  
    opcode = (decrypted_packet[1] >> 4) & 0x0F
```

- Se a trama recebida tiver um tamanho de apenas 2 bytes, é o porque está a haver comunicação. Verifica qual é o comando (se o **opcode** é o 9,(1001)) e avisa que uma mensagem foi recebida com sucesso pela rede.